

Esempi di Sincronizzazione



Classici Problemi di Sincronizzazione

- Problemi classici usati per testare schemi di sincronizzazione
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem

Bounded-Buffer Problem

- n buffer, ognuno può contenere un articolo
- Semaforo **mutex** inizializzato al valore 1
- Semaforo **full** inizializzato al valore 0
- Semaforo **empty** inizializzato al valore n

Bounded Buffer Problem

- La struttura del produttore

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
} while (true);
```

Bounded Buffer Problem

- La struttura del consumatore

```
Do {  
    wait(full);  
    wait(mutex);  
  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```

Readers-Writers Problem

- Un insieme di dati è condiviso tra n processi concorrenti
 - Readers – leggono solo i dati; **non** aggiornano
 - Writers – leggono e scrivono
- **Problema** – permettere a lettori multipli di leggere allo stesso tempo
 - solo un singolo scrittore può accedere ai dati condivisi allo stesso tempo
- Diverse variazioni – basate su priorità
- Dati condivisi:
 - Data set
 - Semaforo binario **rw_mutex** inizializzato a 1
 - Semaforo binario **mutex** inizializzato a 1
 - ▶ protegge **read_count**
 - Intero **read_count** inizializzato a 0

Readers-Writers Problem

□ Struttura scrittore

```
do {  
    wait(rw_mutex);  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex);  
} while (true);
```

Readers-Writers Problem

□ Struttura del lettore

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw_mutex

Readers-Writers Problem

□ Struttura del lettore

Primo lettore 

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw_mutex

Readers-Writers Problem

□ Struttura del lettore

Altri lettori in attesa  `do {`

Primo lettore aspetta  `wait(mutex);`

`read_count++;`

`if (read_count == 1)`

`wait(rw_mutex);`

`signal(mutex);`

`...`

`/* reading is performed */`

`...`

`wait(mutex);`

`read_count--;`

`if (read_count == 0)`

`signal(rw_mutex);`

`signal(mutex);`

`} while (true);`

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw_mutex

Readers-Writers Problem

□ Struttura del lettore

```
do {  
    Altro lettore  wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        Primo lettore  wait(rw_mutex);  
        prende il lock di rw_mutex  
        signal(mutex);  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

Il primo lettore prende il lock di rw_mutex e permette l'accesso agli altri lettori

Readers-Writers Problem

□ Struttura del lettore

Altri lettori evitano
il wait su `rw_mutex` e
si alternano solo su
`mutex`



```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);          sblocco  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

Sbloccato dal primo
lettore gli altri lettori
evitano la wait su
`rw_mutex`

Tutti i lettori in mutua
esclusione su
`read_count` usando
`mutex`

Readers-Writers Problem

□ Struttura del lettore

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);          sblocco  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```

Ultimo lettore
lascia il `rw_mutex`



L'ultimo lettore rilascia
`rw_mutex` quindi o scrittore
o primo lettore potrà
prenderlo

Readers-Writers Problem

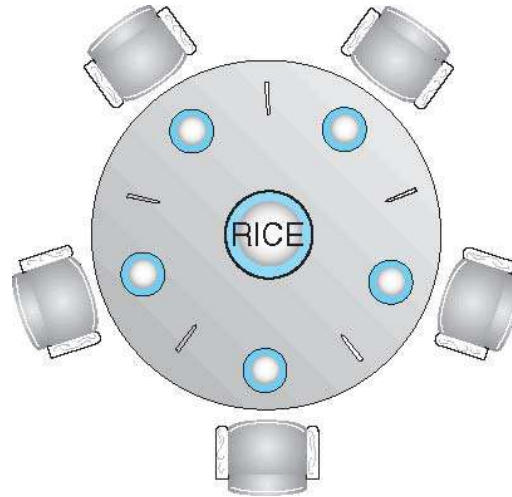
- **Read-write lock** introdotti per risolvere questo tipo di problema:
 - Lock in read mode o write mode
 - Più reader in possono prendere in read mode
 - Un solo writer in write mode

- Utili:
 - ▶ Quando facile identificare quali processi leggono solo dati condivisi e quali processi scrivono solo dati condivisi.
 - ▶ Quando si hanno più lettori che scrittori.
 - I read-write lock generalmente richiedono più overhead dei semafori o mutex lock
 - Consentire il parallelismo di più lettori compensa il sovraccarico

Problemi Readers-Writers Variazioni

- ❑ **Primo problema** – nuovi lettori non rilasciano l'accesso a un scrittore che arriva prima (lo scrittore entra solo quando escono tutti i lettori)
- ❑ **Secondo problema** – una volta che lo scrittore è pronto esegue la scrittura ASAP (se uno scrittore è pronto i lettori successive attendono)
- ❑ In entrambi i casi si può avere starvation portando anche a più variazioni (nel primo caso lo scrittore può attendere troppo, nel secondo i lettori)
- ❑ In alcuni sistemi il kernel fornisce un reader-writer lock

Problema della Cena dei Filosofi



- Filosofi pensano e mangiano
- Non interagiscono con i vicini, a volte provano a prendere 2 bacchette (una alla volta) per mangiare
 - Due per mangiare, poi le rilasciano quando hanno finito
- Se 5 filosofi
 - Dati condivisi
 - ▶ Ciotola di riso (data set)
 - ▶ Semaforo **chopstick** [5] inizializzato a 1

Problema della Cena dei Filosofi

□ filosofo *i*:

do {

wait (chopstick[i]);

wait (chopStick[(i + 1) % 5]);

// eat

signal (chopstick[i]);

signal (chopstick[(i + 1) % 5]);

// think

} while (TRUE);

□ Problema di questo algoritmo?

Problema della Cena dei Filosofi

□ filosofo *i*:

```
do {
```

```
    wait (chopstick[i] );
```

```
    wait (chopStick[ (i + 1) % 5] );
```

```
        // eat
```

```
    signal (chopstick[i] );
```

```
    signal (chopstick[ (i + 1) % 5] );
```

```
        // think
```

```
    } while (TRUE);
```

□ Problema di questo algoritmo?

□ Se tutti i filosofi prendono contemporaneamente la bacchetta sinistra deadlock

Problema della Cena dei Filosofi

□ filosofo *i*:

```
do {
```

```
    wait (chopstick[i] );
```

```
    wait (chopStick[ (i + 1) % 5] );
```

```
        // eat
```

```
    signal (chopstick[i] );
```

```
    signal (chopstick[ (i + 1) % 5] );
```

```
        // think
```

```
    } while (TRUE);
```

□ Rimedi:

- Permetti di sedere solo a 4 filosofi

- Permetti di prendere le bacchette solo se sono entrambe disponibili

- Soluzione asimmetrica (numero dispari di filosofi sx e numero pari dx)

Soluzione con Monitor

```
monitor DiningPhilosophers
```

```
{
```

```
    enum { THINKING, HUNGRY, EATING} state [5] ;
```

```
    condition self [5];
```

stati usati per
coordinare le azioni

```
    void pickup (int i) {
```

```
        state[i] = HUNGRY;
```

```
        test(i);
```

```
        if (state[i] != EATING) self[i].wait;
```

test valuta e cambia
stato in EATING

```
    }
```

Self[i] Permette ad *i* di
ritardare quando è
affamato ma non può
prendere entrambe le
bacchette.

```
    void putdown (int i) {
```

```
        state[i] = THINKING;
```

```
        // test left and right neighbors
```

```
        test((i + 4) % 5);
```

```
        test((i + 1) % 5);
```

```
}
```

Un filosofo alla volta mangia (quando ha fame) prendendo entrambe le bacchette. Se ha fame controlla prima se ci sono entrambe le bacchette.

Problema della Cena dei Filosofi

```
void test (int i) {  
    if ((state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}  
  
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}  
}
```

Verifica se i vicini di i mangiano nel caso sia affamato. Se non mangiano si setta in EATING. Dopo il controllo sblocca i

Soluzione Cena Filosofi

- Ogni filosofo *i* invoca `pickup()` e `putdown()`

`DiningPhilosophers.pickup(i);`

`EAT`

`DiningPhilosophers.putdown(i);`

- Non deadlock, ma starvation è possibile